

Zu Beginn wird das Schieberegister zurückgesetzt. Danach werden die einzelnen Bits der Zahl 21 mit jedem Takt an das Schieberegister übertragen. Dabei ist darauf zu achten, dass die Zeit zwischen den Signalwechsel lang genug ist.

```

1  /* ET_48_74HC164_Test
2  *
3  * Pinbelegung
4  *
5  * Funktion | 74164 | Arduino | MC
6  * Neg CLR  | 9      | A1      | PC0
7  * Data     | 1, 2    | A2      | PC1
8  * CLK      | 8      | A3      | PC2
9  */
10
11 #define PIN_NEG_CLR 0
12 #define PIN_DATA 1
13 #define PIN_CLK 2
14 #define THIRD_CLK_MS 10
15
16 // 21 entspricht 0100 1000
17 const byte data = 0x48;
18
19 void transmit_data() {
20     PORTC &= ~(1 << PIN_NEG_CLR);
21     delay(THIRD_CLK_MS);
22
23     PORTC |= 1 << PIN_NEG_CLR;
24     delay(THIRD_CLK_MS);
25
26     for (uint8_t bit_index = 0; bit_index < 8; bit_index++) {
27         // Serial data bit
28         const byte data_bit = (data >> bit_index) & 0x01;
29
30         // Clear PIN_DATA then set the data_bit
31         PORTC = (PORTC & ~(1 << PIN_DATA)) | data_bit << PIN_DATA;
32         delay(THIRD_CLK_MS);
33
34         // Clock high
35         PORTC |= 1 << PIN_CLK;
36         Serial.println("Clock high");
37         delay(THIRD_CLK_MS);
38
39         // Clock low
40         PORTC &= ~(1 << PIN_CLK);
41         Serial.println("Clock low");
42         delay(THIRD_CLK_MS);
43     }
44
45     PORTB |= 1 << PB5;
46 }
47
48 void setup() {
49     Serial.begin(9600);
50
51     // Use onboard LED
52     DDRB = 1 << PB5;

```

```

53     DDRC = 0x07;
54     PORTC = 1 << PIN_NEG_CLR;
55
56     transmit_data();
57 }
58
59
60 void loop() {
61 }

```

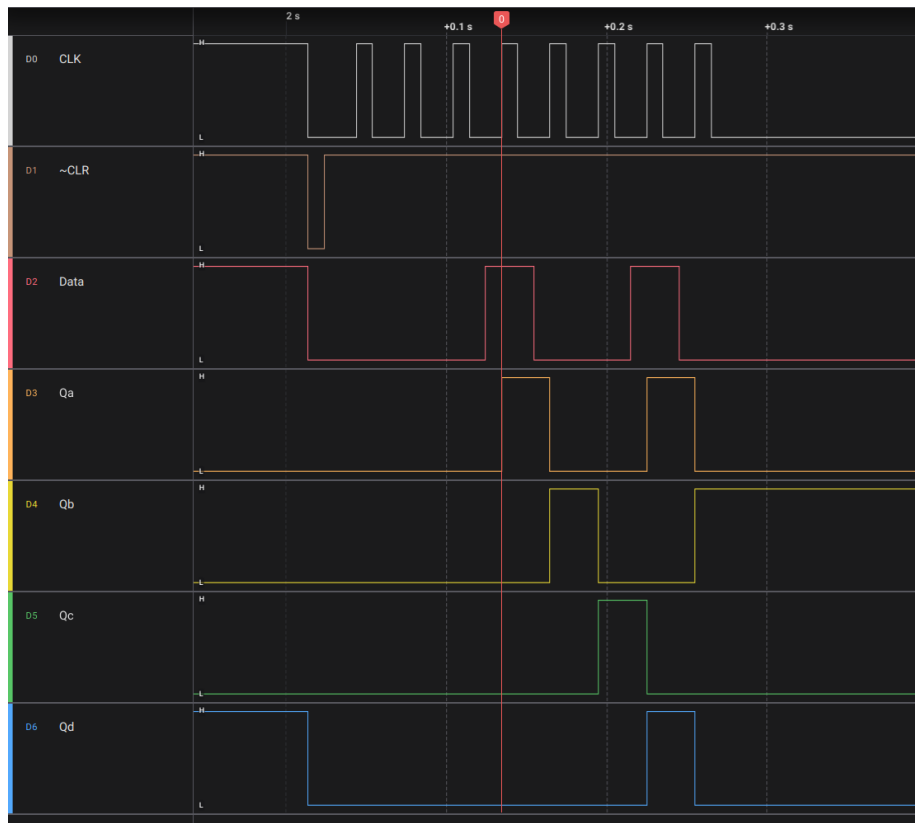


Abbildung 1: Messung von Qa bis Qd

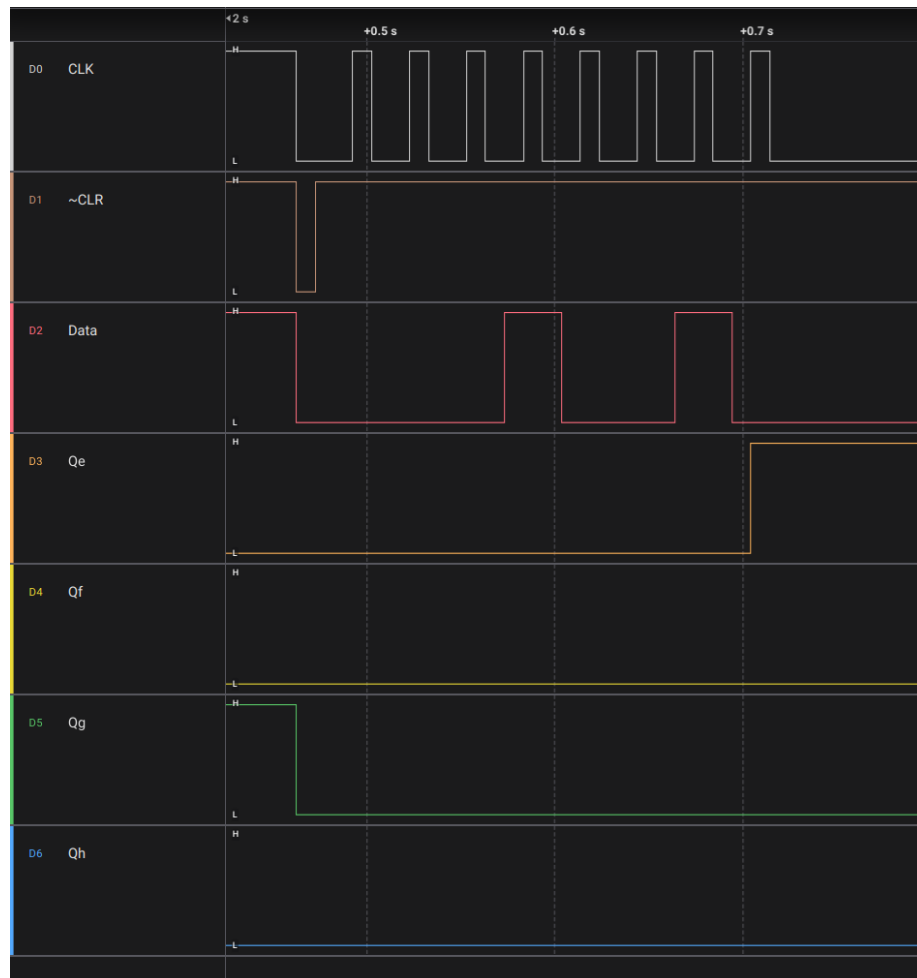


Abbildung 2: Messung von Qe bis Qh